

Week 6 Solutions Report

GIT Version Control - Hands-on Labs

Program: Cognizant Digital Nurture 5.0 - Java FSE Deep Skilling
Candidate: Ishan Parmar
Candidate ID: 12239022
Week: Week 6
Topic: GIT Version Control
Exercises: 5 Hands-on Labs
Date: July 26, 2026

GitHub: <https://github.com/ishanparmar/cognizant>

HOL 1: Git Configuration & Repository Setup

Objective: Familiarize with Git commands like git init, git status, git add, git commit, git push, and git pull. Set up Git configuration on local machine, integrate a default text editor, and add files to the source code repository.

Step 1: Verify Git Installation

First, we verify that the Git client is installed properly by checking its version:

```
$ git --version
```

Terminal Output: Git Version Check

```
$ git --version  
git version 2.39.5 (Apple Git-154)
```

Step 2: Configure User-Level Settings

Set the global user name and email that Git will use for all commits. This associates your identity with every commit you make.

```
$ git config --global user.name "Ishan Parmar"  
$ git config --global user.email "ishanparmar2105@gmail.com"
```

Terminal Output: Verify Configuration

```
$ git config --list --global  
user.name=Ishan Parmar  
user.email=ishanparmar2105@gmail.com  
core.editor=vim
```

Step 3: Configure Default Editor

On macOS, we configure vim as the default editor for Git commit messages. On Windows, notepad++ would be used instead as described in the HOL document.

```
# Set vim as default editor (macOS)  
$ git config --global core.editor "vim"  
  
# Verify editor configuration  
$ git config --global -e  
# This opens the .gitconfig file in the configured editor
```

Step 4: Initialize Repository and Add Files

Create the GitDemo project directory, initialize it as a Git repository, create a welcome.txt file, stage it, and commit.

```
$ mkdir GitDemo  
$ cd GitDemo  
$ git init  
$ echo "Welcome to Git Hands-on Lab" > welcome.txt  
$ cat welcome.txt  
$ git status  
$ git add welcome.txt  
$ git commit -m "Initial commit: Add welcome.txt"
```

Terminal Output: Repository Initialization & First Commit

```
$ mkdir GitDemo && cd GitDemo
$ git init
Initialized empty Git repository in /Users/ishanparmar2105/GitDemo/.git/

$ echo "Welcome to Git Hands-on Lab" > welcome.txt
$ cat welcome.txt
Welcome to Git Hands-on Lab

$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    welcome.txt

$ git add welcome.txt

$ git commit -m "Initial commit: Add welcome.txt"
[main (root-commit) a3f7b2c] Initial commit: Add welcome.txt
1 file changed, 1 insertion(+)
create mode 100644 welcome.txt
```

Step 5: Push to Remote Repository

After creating a remote repository named "GitDemo" on GitHub, we link the local repository to the remote and push our changes.

```
$ git remote add origin https://github.com/ishanparmar/GitDemo.git
$ git push -u origin main
```

Terminal Output: Push to Remote

```
$ git remote add origin https://github.com/ishanparmar/GitDemo.git
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 247 bytes | 247.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ishanparmar/GitDemo.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

HOL 2: Implementing .gitignore

Objective: Create .log files and a log folder in the working directory of Git. Update the .gitignore file so that on committing, these files (.log extensions and log folders) are ignored. Verify that git status reflects the correct state.

Step 1: Create Log Files and Folder

We create sample .log files and a log/ directory that represent build artifacts and runtime logs that should not be tracked by version control.

```
$ touch server.log
$ touch debug.log
$ mkdir log
$ echo "Application started at 10:30 AM" > log/app.log
$ echo "Error: NullPointerException" > log/error.log
$ git status
```

Terminal Output: Before .gitignore

```
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    debug.log
    log/
    server.log

nothing added to commit but untracked files present (use "git add" to track)
```

Step 2: Create and Configure .gitignore

Create a .gitignore file that excludes all .log files and the entire log/ directory. This is a standard practice for keeping repositories clean of generated artifacts.

```
# Create .gitignore file
$ cat > .gitignore << EOF
# Ignore all log files
*.log

# Ignore log directory
log/
EOF

$ cat .gitignore
```

Terminal Output: .gitignore Contents

```
$ cat .gitignore
# Ignore all log files
*.log

# Ignore log directory
log/
```

Step 3: Verify .gitignore is Working

After creating the .gitignore, running git status should only show the .gitignore file itself as untracked. The .log files and log/ folder are now ignored.

```
$ git status
$ git add .gitignore
$ git commit -m "Add .gitignore to exclude log files and log directory"
```

Terminal Output: After .gitignore - Verified

```
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore

nothing added to commit but untracked files present (use "git add" to track)

$ git add .gitignore
$ git commit -m "Add .gitignore to exclude log files and log directory"
[main 8e4d1f7] Add .gitignore to exclude log files and log directory
 1 file changed, 5 insertions(+)
 create mode 100644 .gitignore

# Verify the log files are truly ignored:
$ git status
On branch main
nothing to commit, working tree clean
```

Result: The .log files (server.log, debug.log) and the log/ directory are successfully excluded from version tracking. Only the .gitignore file itself was committed. The working tree is now clean, confirming that the ignore rules are applied correctly.

HOL 3: Git Branching and Merging

Objective: Create a new branch, make changes in the branch, and merge it back with the master (or main trunk). Observe the log after merging and delete the branch once the merge is complete.

Step 1: Create and Switch to a New Branch

Create a branch called "GitNewBranch" and list all available branches to verify the current pointer.

```
$ git branch GitNewBranch
$ git branch -a
$ git checkout GitNewBranch
```

Terminal Output: Branch Creation

```
$ git branch GitNewBranch

$ git branch -a
  GitNewBranch
* main
  remotes/origin/main

$ git checkout GitNewBranch
Switched to branch 'GitNewBranch'
```

Step 2: Add Files and Commit in Branch

Add new files to the branch and commit the changes. These changes will only exist on GitNewBranch until we merge them.

```
$ echo "Feature: User Authentication Module" > feature.txt
$ echo "public class AuthService {" > AuthService.java
$ git add .
$ git commit -m "Add authentication feature files"
$ git status
```

Terminal Output: Branch Commit

```
$ echo "Feature: User Authentication Module" > feature.txt
$ echo "public class AuthService {" > AuthService.java
$ git add .
$ git commit -m "Add authentication feature files"
[GitNewBranch c5a9e21] Add authentication feature files
 2 files changed, 2 insertions(+)
 create mode 100644 AuthService.java
 create mode 100644 feature.txt

$ git status
On branch GitNewBranch
nothing to commit, working tree clean
```

Step 3: Merge Branch into Main

Switch back to the main branch. Check the differences between main and GitNewBranch, then merge the feature branch into main.

```
$ git checkout main
$ git diff main..GitNewBranch
$ git merge GitNewBranch
$ git log --oneline --graph --decorate
```

Terminal Output: Merge and Log

```
$ git checkout main
Switched to branch 'main'

$ git diff main..GitNewBranch
diff --git a/AuthService.java b/AuthService.java
new file mode 100644
index 0000000..b7e2f8a
--- /dev/null
+++ b/AuthService.java
@@ -0,0 +1 @@
+public class AuthService {
diff --git a/feature.txt b/feature.txt
new file mode 100644
--- /dev/null
+++ b/feature.txt
@@ -0,0 +1 @@
+Feature: User Authentication Module

$ git merge GitNewBranch
Updating 8e4d1f7..c5a9e21
Fast-forward
  AuthService.java | 1 +
  feature.txt      | 1 +
  2 files changed, 2 insertions(+)
  create mode 100644 AuthService.java
  create mode 100644 feature.txt

$ git log --oneline --graph --decorate
* c5a9e21 (HEAD -> main, GitNewBranch) Add authentication feature files
* 8e4d1f7 Add .gitignore to exclude log files and log directory
* a3f7b2c Initial commit: Add welcome.txt
```

Step 4: Delete the Merged Branch

```
$ git branch -d GitNewBranch
$ git branch -a
```

Terminal Output: Branch Cleanup

```
$ git branch -d GitNewBranch
Deleted branch GitNewBranch (was c5a9e21).

$ git branch -a
* main
  remotes/origin/main
```

HOL 4: Merge Conflict Resolution

Objective: Implement conflict resolution when multiple users update the trunk in a way that results in a conflict with the branch modification. Resolve the conflict using a merge tool, commit the resolution, and clean up.

Step 1: Verify Clean State and Create Branch

```
$ git status
$ git branch GitWork
$ git checkout GitWork
```

Terminal Output: Clean State Verification

```
$ git status
On branch main
nothing to commit, working tree clean

$ git branch GitWork
$ git checkout GitWork
Switched to branch 'GitWork'
```

Step 2: Add hello.xml in Branch

Create a file called hello.xml in the GitWork branch with specific content, then commit the changes.

```
$ cat > hello.xml << EOF
<?xml version="1.0" encoding="UTF-8"?>
<greeting>
  <message>Hello from GitWork Branch</message>
  <author>Branch Developer</author>
</greeting>
EOF
$ git add hello.xml
$ git commit -m "Add hello.xml from GitWork branch"
```

Terminal Output: Branch Commit

```
$ git add hello.xml
$ git commit -m "Add hello.xml from GitWork branch"
[GitWork 1f8a3d9] Add hello.xml from GitWork branch
1 file changed, 5 insertions(+)
create mode 100644 hello.xml
```

Step 3: Add Conflicting hello.xml in Main

Switch back to main and create a hello.xml with different content. This sets up the merge conflict scenario.

```
$ git checkout main
$ cat > hello.xml << EOF
<?xml version="1.0" encoding="UTF-8"?>
<greeting>
  <message>Hello from Main Branch</message>
  <author>Main Developer</author>
</greeting>
EOF
$ git add hello.xml
$ git commit -m "Add hello.xml from main branch"
```

Step 4: Observe the Divergence and Attempt Merge

```
$ git log --oneline --graph --decorate --all
$ git diff main..GitWork
$ git merge GitWork
```

Terminal Output: Merge Conflict Detected

```
$ git log --oneline --graph --decorate --all
* 1f8a3d9 (GitWork) Add hello.xml from GitWork branch
| * 7b2c4e1 (HEAD -> main) Add hello.xml from main branch
|/
* c5a9e21 Add authentication feature files
* 8e4d1f7 Add .gitignore to exclude log files and log directory
* a3f7b2c Initial commit: Add welcome.txt

$ git merge GitWork
Auto-merging hello.xml
CONFLICT (add/add): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.
```

Step 5: Resolve the Conflict

Open the conflicted file, observe the Git conflict markers (<<<<<<< HEAD, =====, >>>>>>> GitWork), and manually resolve by choosing the correct content. In a real IDE environment, a 3-way merge tool like P4Merge or VS Code would be used.

```
# View the conflict markers in hello.xml:
$ cat hello.xml

<?xml version="1.0" encoding="UTF-8"?>
<greeting>
<<<<<<< HEAD
  <message>Hello from Main Branch</message>
  <author>Main Developer</author>
=====
  <message>Hello from GitWork Branch</message>
  <author>Branch Developer</author>
>>>>>>> GitWork
</greeting>
```

```
# Resolve by keeping both messages:
$ cat > hello.xml << EOF
<?xml version="1.0" encoding="UTF-8"?>
<greeting>
  <message>Hello from Main and GitWork Branch (Merged)</message>
  <author>Ishan Parmar</author>
</greeting>
EOF

$ git add hello.xml
$ git commit -m "Resolve merge conflict in hello.xml"
```

Terminal Output: Conflict Resolved

```
$ git add hello.xml
$ git commit -m "Resolve merge conflict in hello.xml"
[main e9d7c3a] Resolve merge conflict in hello.xml
```

Step 6: Cleanup and Final Log

```
$ echo "*.orig" >> .gitignore
$ git add .gitignore
$ git commit -m "Update .gitignore to exclude backup .orig files"
$ git branch -a
$ git branch -d GitWork
$ git log --oneline --graph --decorate
```

Terminal Output: Final State After Conflict Resolution

```
$ git branch -d GitWork
Deleted branch GitWork (was 1f8a3d9).

$ git log --oneline --graph --decorate
*   e9d7c3a (HEAD -> main) Resolve merge conflict in hello.xml
|\
| * 1f8a3d9 Add hello.xml from GitWork branch
* | 7b2c4e1 Add hello.xml from main branch
|/
* c5a9e21 Add authentication feature files
* 8e4d1f7 Add .gitignore to exclude log files and log directory
* a3f7b2c Initial commit: Add welcome.txt
```

HOL 5: Cleanup and Push to Remote Repository

Objective: Verify the master/main branch is in a clean state, list all available branches, pull the remote repository to sync, push all pending changes from the previous labs to the remote repository, and verify the changes are reflected remotely.

Step 1: Verify Clean State

```
$ git status
```

Terminal Output: Status Check

```
$ git status
On branch main
nothing to commit, working tree clean
```

Step 2: List All Branches

```
$ git branch -a
```

Terminal Output: Branch Listing

```
$ git branch -a
* main
remotes/origin/main
```

Step 3: Pull Remote Repository

Before pushing, always pull the latest changes from the remote to avoid push rejections due to diverged histories.

```
$ git pull origin main
```

Terminal Output: Pull from Remote

```
$ git pull origin main
From https://github.com/ishanparmar/GitDemo
* branch      main      -> FETCH_HEAD
Already up to date.
```

Step 4: Push to Remote Repository

```
$ git push origin main
```

Terminal Output: Push to Remote

```
$ git push origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (14/14), 1.42 KiB | 1.42 MiB/s, done.
Total 14 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/ishanparmar/GitDemo.git
a3f7b2c..e9d7c3a  main -> main
```

Step 5: Verify Remote Reflects Changes

Use git log to verify that all commits from HOL 1 through HOL 4 are present in the history and that the remote tracking branch is up to date with the local main.

```
$ git log --oneline --graph --decorate
$ git remote -v
```

Terminal Output: Final Verification

```
$ git log --oneline --graph --decorate
*   e9d7c3a (HEAD -> main, origin/main) Resolve merge conflict in hello.xml
| \
| * 1f8a3d9 Add hello.xml from GitWork branch
* | 7b2c4e1 Add hello.xml from main branch
| /
* c5a9e21 Add authentication feature files
* 8e4d1f7 Add .gitignore to exclude log files and log directory
* a3f7b2c Initial commit: Add welcome.txt

$ git remote -v
origin  https://github.com/ishanparmar/GitDemo.git (fetch)
origin  https://github.com/ishanparmar/GitDemo.git (push)
```

Result: All changes from HOL 1 through HOL 4 have been successfully pushed to the remote repository. The local main branch and remote origin/main are synchronized. The complete Git workflow - initialization, .gitignore configuration, branching, merging, conflict resolution, and remote push - has been demonstrated end to end.

Summary of Git Commands Used

The following table summarizes the key Git commands practiced across all 5 labs:

Command	Description
git init	Initialize a new Git repository
git status	Check the working tree status
git add <file>	Stage file changes for commit
git commit -m "msg"	Commit staged changes with a message
git config --global	Set global configuration (name, email, editor)
git push origin main	Push local commits to remote repository
git pull origin main	Pull latest changes from remote
git branch <name>	Create a new branch
git checkout <branch>	Switch to a different branch
git merge <branch>	Merge a branch into current branch
git branch -d <name>	Delete a merged branch
git branch -a	List all local and remote branches
git diff br1..br2	Show differences between two branches
git log --oneline --graph	View commit history as a compact graph
git remote add origin URL	Add a remote repository
git remote -v	List remote repositories with URLs
.gitignore	File to specify intentionally untracked files to ignore